FORMATION DÉVELOPPEMENT WEB

Jour 2

Introduction au back-end

Navigateur, Laravel et PostgreSQL

Première séquence : 25 minutes



Objectifs de la séquence

Comprendre le rôle de chaque partie de notre application

01 Situer le point de départ

Expliquer ce que l’interface du jour 1 sait faire et ce qui disparaît au rechargement.

02 Identifier les composants

Distinguer le navigateur, l’application Laravel et la base PostgreSQL.

03 Suivre une demande

Décrire le trajet des informations lorsqu’un document est ajouté puis retrouvé.

Notre point de départ

L’application « Mes documents » du jour 1

Ce que l’utilisateur voit

Une liste de documents, un formulaire et une interface adaptée aux écrans.

Ce que JavaScript réalise

Il vérifie les champs et ajoute une ligne dans la page du navigateur.

La limite actuelle

Un rechargement efface les ajouts. Aucun document ajouté ni fichier sélectionné n’est envoyé à un serveur.

Pourquoi ajouter un back-end ?

Conserver et retrouver les informations des documents

Le besoin

Retrouver le titre et la catégorie après avoir fermé ou rechargé la page.

Le rôle du back-end

Recevoir une demande, vérifier les données et décider du traitement à effectuer.

Le rôle de la base de données

Conserver les informations pour que le serveur puisse les retrouver plus tard.

Les composants de l’application

Le navigateur échange avec Laravel, qui communique avec PostgreSQL



Laravel reçoit aussi le résultat de la base et renvoie une réponse au navigateur.

Chaque composant a un rôle différent, même sur un seul ordinateur.

Le navigateur et le front-end

La partie avec laquelle l’utilisateur interagit

Dans notre projet

Le navigateur affiche le HTML et le CSS. JavaScript réagit aux actions de l’utilisateur.

Lorsque le formulaire est envoyé

L’interface recueille le titre et la catégorie, puis pourra demander au serveur de les enregistrer.

La réponse

Le navigateur pourra afficher la liste actualisée ou un message d’erreur.

Laravel et le back-end

Le code PHP qui traite les demandes

Laravel

Un framework PHP qui aide à organiser le code de l’application côté serveur.

Son travail dans le fil rouge

Recevoir la demande, contrôler les informations, puis lire ou enregistrer des données.

Son résultat

Renvoyer une réponse que le navigateur pourra utiliser pour mettre la page à jour.

PostgreSQL et la base de données

Un système pour conserver et retrouver des données

Une table de documents

Elle pourra contenir un identifiant, un titre, une catégorie et des dates.

Une ligne par document

Exemple : 1, « Support de cours », « Formation ». Ces valeurs représentent des informations sur un document.

Le lien avec Laravel

Laravel demande à PostgreSQL d’enregistrer ou de retrouver ces informations.

Ajouter un document

Le trajet prévu pour le titre et la catégorie



1 Navigateur

L’utilisateur saisit le titre et la catégorie.

2 Laravel

Le serveur reçoit et vérifie les données.

3 PostgreSQL

Laravel demande leur enregistrement.

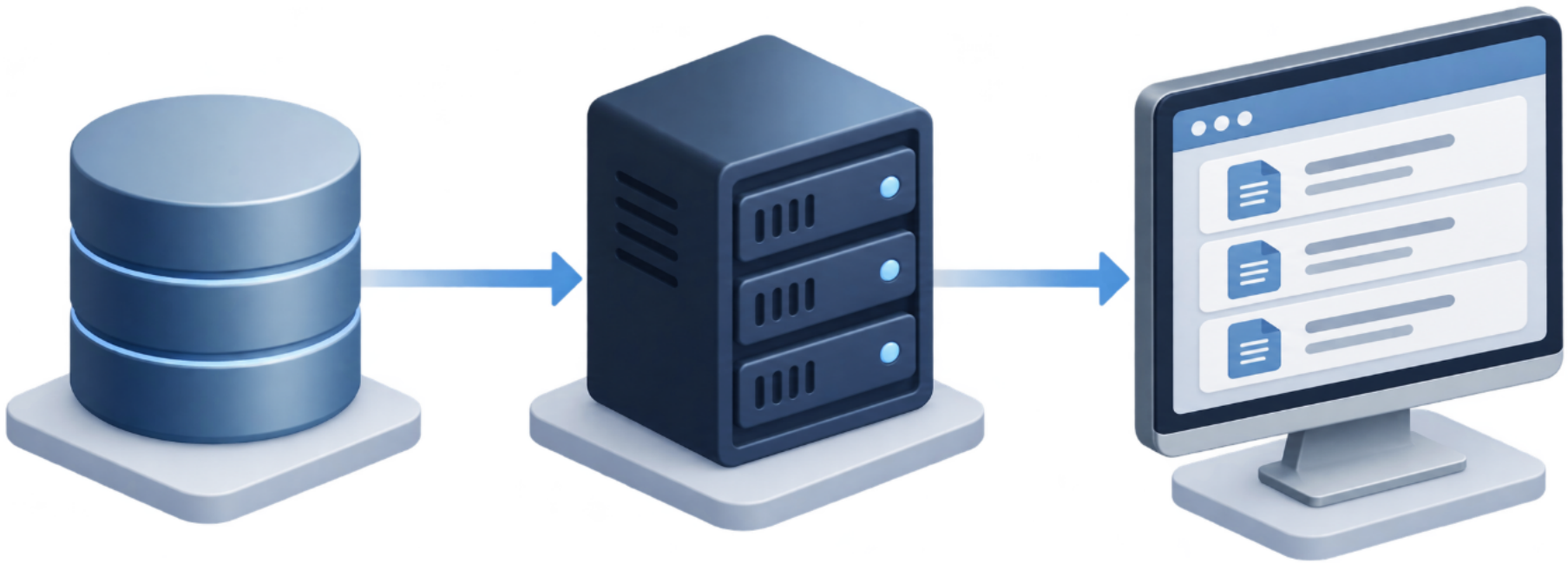
4 Réponse

Laravel confirme l’ajout ou signale une erreur.

À cette étape, seul le titre et la catégorie sont transmis : le fichier n’est pas envoyé.

Retrouver les documents

Ce qui se passe au prochain chargement de la page



L’illustration montre le retour des données vers le navigateur.

1 Demande

Au chargement, le navigateur demande la liste à Laravel.

2 Lecture

Laravel retrouve les informations enregistrées dans PostgreSQL.

3 Affichage

Laravel répond au navigateur, qui construit la liste à l’écran.

Le navigateur communique avec Laravel, qui accède à la base.

Application locale et stockage

Où se trouvent les composants pendant la formation ?

Sur le poste de travail

Le navigateur affiche l’interface. Laravel et PostgreSQL fonctionnent aussi localement, selon l’environnement préparé.

Des rôles distincts

Le navigateur affiche et demande. Laravel traite. PostgreSQL conserve les données.

Le fichier sélectionné

Pour cette étape, nous travaillons sur les informations du document. Le téléversement sera abordé plus tard.

Exercice de compréhension

Suivre l’ajout d’un document sans écrire de code

Une personne saisit « Contrat de formation » et choisit « Administratif ».

Question 1

Quel composant affiche le formulaire et récupère les valeurs saisies ?

Question 2

Quel composant contrôle les données et demande leur enregistrement ?

Question 3

Où retrouver les informations après un rechargement si elles ont été enregistrées ?

Correction et transition

Les trois rôles à retenir

Navigateur

Il affiche le formulaire et envoie les informations à Laravel.

Laravel

Il contrôle les données, communique avec PostgreSQL et renvoie une réponse.

PostgreSQL

Il conserve les informations enregistrées, que Laravel peut retrouver.

Suite : découvrir l’environnement Docker du projet.

Pourquoi utiliser Docker ?

Un environnement commun pour les travaux pratiques

Un environnement reproductible

Chaque participant utilise les mêmes composants et une configuration préparée.



Des outils dans les conteneurs

PHP, Composer et PostgreSQL fonctionnent dans Docker. VS Code reste sur le poste.

Une préparation en amont

Les images et dépendances peuvent nécessiter des téléchargements au premier lancement.

Image, conteneur et volume

Trois notions à distinguer

Image

Un modèle contenant les fichiers et outils nécessaires. Exemple : une image PHP.

Conteneur

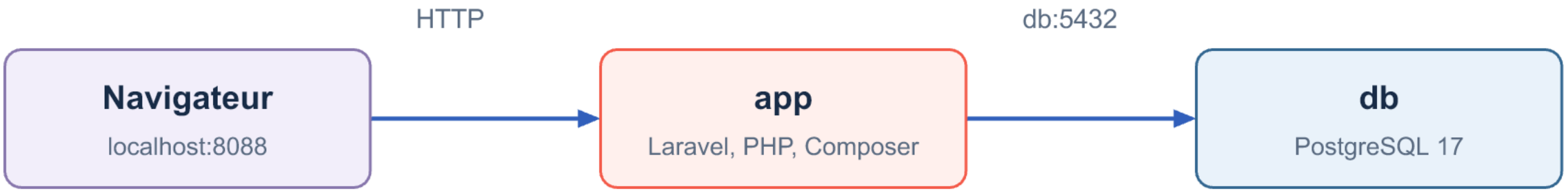
Une instance créée à partir d’une image. Notre projet utilise les services app et db.

Volume

Un espace géré par Docker pour conserver les données PostgreSQL entre les redémarrages.

Notre environnement de formation

VS Code modifie le code ; Docker exécute l'application



Le code est partagé avec app. Le volume postgres_data conserve la base.

Le port 8088 est réservé à cette application sur la machine.

Créer le projet Laravel

Ce que nous avons réalisé pour préparer le projet de départ

1 Utiliser Composer dans Docker

Nous avons lancé un conteneur Composer pour générer un projet Laravel 13.

2 Préparer le projet

Nous avons ajouté le Dockerfile, Compose et la configuration PostgreSQL.

3 Terminer l’installation

Nous avons installé les dépendances et généré la clé depuis le conteneur app.

Commande de création du projet

Exemple PowerShell exécuté depuis le dossier apps

```
$appsPath = (Get-Location).Path
docker run --rm `
  --mount "type=bind,source=$appsPath,target=/work" `
  --workdir /work composer:2 create-project `
  laravel/laravel app-laravel-postgresql "^13.0" `
  --no-interaction --prefer-dist --no-scripts
```

Composer crée le squelette Laravel dans apps/app-laravel-postgresql.

Après récupération du projet, on passe directement au démarrage Docker.

Dockerfile et Docker Compose

Deux fichiers pour construire et démarrer l'environnement

Dockerfile

Décrit l'image app : PHP 8.5, Composer et l'extension PDO PostgreSQL.

compose.yaml

Décrit les services app et db, leur réseau et leurs paramètres de démarrage.

Un démarrage coordonné

Compose attend que PostgreSQL soit prêt avant de démarrer app.

Préparer le poste Windows

À vérifier avant le premier lancement

Environnement

Docker Desktop avec WSL 2, virtualisation activée et Docker Compose fonctionnel.

Outils et ressources

VS Code et Git si le projet est cloné. Prévoir de l’espace disque et le port 8088 libre.

Accès réseau

Préparer les images et les dépendances à l’avance en cas d’environnement isolé.

Démarrer les conteneurs

Dans un terminal ouvert dans apps/app-laravel-postgresql

docker compose up --build -d

up

Crée et démarre les services déclarés dans compose.yaml.

--build et -d

Construit l'image app avant le démarrage et laisse les conteneurs en arrière-plan.

Au premier lancement, app attend encore les dépendances Composer.

Installer les dépendances

Composer s’exécute dans le service app

docker compose exec app composer install

exec app

Exécute la commande dans le conteneur du service app.

composer install

Installe les dépendances du projet à partir de composer.lock.

Aucune installation de Composer sur Windows n’est nécessaire.

Configurer Laravel au premier démarrage

Créer .env et la clé uniquement pour une nouvelle installation

docker compose exec app cp .env.example .env

docker compose exec app php artisan key:generate

Le fichier .env

Il contient la configuration locale, notamment l'adresse de la base db.

La clé de l'application

Laravel génère APP_KEY dans .env. Ce fichier n'est pas à publier sur Git.

Sur notre poste, .env et sa clé ont déjà été créés.

Vérifier le fonctionnement

Observer les services et ouvrir la page Laravel

docker compose ps

docker compose exec app php artisan --version

docker compose exec app php artisan db:show

Dans le navigateur

Ouvrir http://localhost:8088. La page d’accueil Laravel doit s’afficher.

Notre installation : Laravel 13 et PostgreSQL 17.

La base et son volume

Conserver les données entre deux démarrages

Service db

PostgreSQL fonctionne dans son conteneur. Laravel le contacte avec le nom db.



Volume postgres_data

Il conserve les fichiers de la base lorsque les conteneurs sont arrêtés ou recréés.

Attention à la suppression

docker compose down -v supprime aussi le volume et les données qu'il contient.

Commandes pour la suite

Démarrer, arrêter et préparer les migrations

docker compose stop

Arrête les conteneurs sans supprimer les données.

docker compose up -d

Redémarre les services déjà configurés.

docker compose exec app php artisan migrate

À utiliser pendant la séquence sur les migrations.

Exercice de compréhension

Retrouver les étapes de notre installation

Question 1

Quel fichier décrit les services app et db ?

Question 2

Où s’exécute Composer lorsque nous lançons docker compose exec app ?

Question 3

Quel élément permet de conserver les données PostgreSQL ?

Correction

Les éléments à retenir

compose.yaml

Il décrit les services, le réseau, le port local et le volume.

Le conteneur app

Il exécute Laravel, PHP et Composer à partir de notre image.

Le volume postgres_data

Il conserve les fichiers de la base. Le fichier .env contient la configuration locale.

Suite du jour 2 : HTTP, JSON et API REST.

HTTP, JSON et API REST

Comprendre les échanges avant de construire notre API

HTTP

Le navigateur envoie une requête et reçoit une réponse du serveur.

JSON

Un format texte qui permet de représenter des données structurées.

API REST

Des adresses et des méthodes HTTP pour manipuler des ressources.

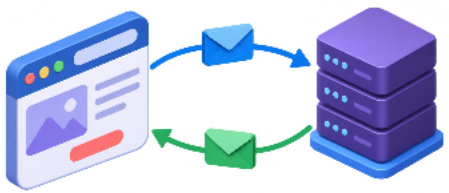
Durée indicative : 35 minutes, exercices compris.

HTTP : une demande et une réponse

Un échange entre le navigateur et le serveur

1 Le navigateur envoie une requête

Exemple : demander la liste des documents à l'application.



2 Le serveur traite la demande

Laravel pourra rechercher les informations nécessaires.

3 Le serveur renvoie une réponse

Elle indique le résultat et peut contenir des données.

Une requête HTTP

Ce que le client envoie au serveur

```
GET /api/documents HTTP/1.1
Host: localhost:8088
Accept: application/json
```

Méthode et chemin

GET demande la ressource située à l'adresse /api/documents.

En-têtes

Host identifie le serveur. Accept indique que le client souhaite du JSON.

Exemple fictif : cette API sera créée plus tard avec Laravel.

Une réponse HTTP

Ce que le serveur renvoie au client

```
HTTP/1.1 200 OK
Content-Type: application/json

[{"id": 12, "title": "Contrat"}]
```

Statut 200

Le serveur indique que la requête a réussi.

En-tête et corps

Content-Type annonce du JSON ; le corps contient ici une liste de documents.

Exemple fictif : cette API sera créée plus tard avec Laravel.

Les méthodes HTTP

La méthode indique l'action demandée sur une ressource

GET	Lire une liste ou un document.
POST	Créer un nouveau document.
PUT	Remplacer les informations d'un document existant.
DELETE	Supprimer un document existant.

La réponse indique si l'action a réussi ou si une erreur est survenue.

GET et POST

Lire des documents ou demander leur création

```
GET /api/documents
GET /api/documents/12
```

Lire toute la liste ou le document identifié par 12.

```
POST /api/documents
Content-Type: application/json

{"title": "Contrat", "category": "Administratif"}
```

Le serveur peut répondre 201 si la création réussit.

Exemple fictif : cette API sera créée plus tard avec Laravel.

PUT et DELETE

Modifier ou supprimer le document identifié par 12

```
PUT /api/documents/12
Content-Type: application/json

{"title": "Contrat signé", "category": "Administratif"}
```

PUT envoie les informations à remplacer pour ce document.

```
DELETE /api/documents/12
```

DELETE demande sa suppression. Une réponse 204 peut confirmer l’action sans corps.

Exemple fictif : cette API sera créée plus tard avec Laravel.

Les codes de réponse HTTP

Le statut résume le résultat de la requête

200 OK	La demande a réussi, par exemple une lecture.
201 Created	Une ressource a été créée.
204	La demande a réussi et la réponse ne contient pas de corps.
404	La ressource demandée est introuvable.

Les codes 2xx indiquent un succès. Les codes 4xx signalent une erreur côté demande.

Comprendre les erreurs

Distinguer une demande incorrecte d’une erreur serveur

400 La requête est mal formée, par exemple un JSON invalide.

422 Les données sont lisibles mais ne respectent pas les règles de validation.

500 Le serveur rencontre une erreur interne inattendue.

Le code aide à comprendre le résultat ; le corps peut préciser l’erreur.

JSON : représenter des données

Un format texte utilisé dans les échanges de données

Un format structuré

JSON organise les données en objets, tableaux et valeurs.



Dans notre application

Le serveur pourra renvoyer un document en JSON au navigateur.

Dans le navigateur

JavaScript pourra lire ces données pour construire la liste affichée.

Lire un objet JSON

Un document décrit par des couples clé et valeur

```
{
  "id": 12,
  "title": "Contrat",
  "category": "Administratif",
  "archived": false
}
```

id	Nombre : 12.
title	Texte : « Contrat ».
archived	Booléen : false.

Les accolades délimitent l’objet ; chaque clé est suivie de deux-points et d’une valeur.

Lire un tableau JSON

Plusieurs documents dans une même réponse

```
[
  {"id": 12, "title": "Contrat"},
  {"id": 13, "title": "Rapport"}
]
```

Les crochets [ ]

Ils délimitent le tableau, qui contient ici deux objets.

Les accolades { }

Elles délimitent chaque objet représentant un document.

Règles essentielles de JSON

Reconnaître un document JSON valide

Clés et textes

Utiliser des guillemets doubles : "title": "Contrat".

Valeurs

Un nombre, true, false et null s’écrivent sans guillemets.

Séparation

Les éléments sont séparés par des virgules, sans virgule après le dernier.

Une API REST

Une interface pour agir sur des ressources

API

Un ensemble de points d’accès permettant à un client de demander des opérations.

Ressource

Dans notre projet, un document est une ressource identifiée par une adresse.

REST et HTTP

La méthode et l’adresse décrivent l’action. Chaque requête apporte les informations nécessaires à son traitement.

Exemple fictif : cette API sera créée plus tard avec Laravel.

Notre future API de documents

Une même ressource, plusieurs opérations

GET	/api/documents : lister les documents.
POST	/api/documents : créer un document.
PUT	/api/documents/12 : remplacer ses informations.
DELETE	/api/documents/12 : demander sa suppression.

GET /api/documents/12 permettra aussi de consulter un seul document.

Exemple fictif : cette API sera créée plus tard avec Laravel.

Exercice 1 : choisir la méthode

Travail individuel puis correction

A Afficher tous les documents

Quelle méthode et quelle adresse utilisent la collection ?

B Ajouter un document

Quelle méthode envoie les nouvelles informations ?

C Modifier le document 12

Quelle méthode et quelle adresse ciblent ce document ?

D Supprimer le document 12

Quelle méthode et quelle adresse utilisent son identifiant ?

Correction de l’exercice 1

Méthodes et adresses de notre future API

A GET /api/documents : obtenir la liste.

B POST /api/documents : demander la création.

C PUT /api/documents/12 : remplacer ses informations.

D DELETE /api/documents/12 : demander la suppression.

Exemple fictif : cette API sera créée plus tard avec Laravel.

Exercice 2 : lire JSON et un statut

Observer les données avant de répondre

```
{"id": 12, "title": "Contrat",
 "category": "Administratif"}
```

A Quelle est la valeur de title ?

B id est-il un nombre ou un texte ?

C Quel code peut indiquer une création réussie ?

D Quel code peut signaler une validation échouée ?

Correction de l’exercice 2

Lire la valeur et interpréter le résultat

A title contient le texte « Contrat ».

B id est le nombre 12 : il n’a pas de guillemets.

C 201 peut indiquer qu’une ressource a été créée.

D 422 peut indiquer que les données ne respectent pas la validation.

À retenir

De la théorie à la première route Laravel

HTTP

Le client envoie une requête et le serveur répond avec un code.

JSON

Les données peuvent être représentées sous forme d’objets et de tableaux.

API REST

La méthode HTTP et l’adresse désignent l’opération sur une ressource.

Prochaine étape : créer une route Laravel qui renvoie du JSON.

Découverte de Laravel

Organisation, configuration, routes et contrôleurs

Se repérer dans le projet

Identifier les dossiers utiles et comprendre le rôle du fichier .env.

Suivre une requête

Comprendre comment une route appelle une méthode de contrôleur.

Construire une réponse JSON

Créer puis tester une route d’exercice avec des données fictives.

Durée indicative : 35 minutes, exercice compris.

Laravel dans notre projet

Un framework PHP pour organiser l’application

Un cadre de travail

Laravel fournit une structure et des outils pour développer une application PHP.



Notre projet

La page d’accueil est fournie par Laravel ; l’API de documents sera développée ensuite.

Notre environnement

Le code est modifié dans VS Code et exécuté dans le conteneur app.

Les dossiers à connaître

Retrouver les éléments utiles pour le jour 2

`routes/`	Les chemins HTTP de l'application.
`app/Http/Controllers/`	Les contrôleurs qui traitent les demandes.
`config/ et .env`	La configuration et les valeurs locales.
`database/migrations/`	Les fichiers qui décrivent les changements de tables.

D’autres éléments du projet

Un fichier ou dossier a un rôle précis

bootstrap/	Configure le démarrage et les fichiers de routes.
app/Models	Contiendra les modèles représentant les données.
vendor/	Contient les dépendances installées par Composer.
tests/	Regroupe les tests de l’application.

Les modèles et les migrations seront étudiés dans la séquence suivante.

Le fichier .env

Des valeurs adaptées à l’environnement local

À quoi sert-il ?

Il fournit des valeurs locales : environnement, URL, clé et connexion à la base.



.env.example

C’est un modèle partagé pour créer le .env d’une nouvelle installation.

Clé et informations sensibles

Ne pas publier le .env ni remplacer la clé d’une installation existante.

La connexion PostgreSQL dans .env

Extrait des valeurs de notre environnement de formation

```
APP_ENV=local
APP_URL=http://localhost:8088
DB_CONNECTION=pgsql
DB_HOST=db
DB_PORT=5432
DB_DATABASE=formation_documents
```

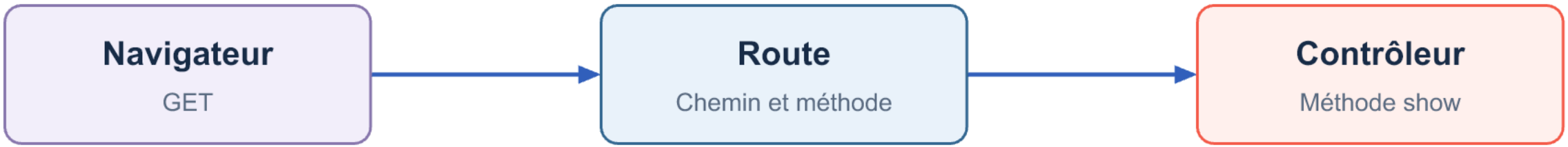
DB_HOST=db

Laravel joint PostgreSQL par son nom de service sur le réseau Docker.

Le mot de passe et APP_KEY restent dans le fichier local et ne sont pas à partager.

Le rôle d’une route

Associer une méthode et une adresse à un traitement



La route indique à Laravel quel traitement exécuter pour la requête.

Le contrôleur retourne ensuite une réponse au navigateur.

Routes web et routes API

Deux fichiers pour des usages différents

routes/web.php

Routes de l’interface web, avec les fonctionnalités du groupe web.

routes/api.php

Routes de notre API, chargées avec le groupe api dans cette application.

Le préfixe /api

Laravel l’ajoute automatiquement au chemin défini dans routes/api.php.

Charger les routes API

Extrait de bootstrap/app.php

```
->withRouting(
    web: __DIR__.'/../routes/web.php',
    api: __DIR__.'/../routes/api.php',
    commands: __DIR__.'/../routes/console.php',
    health: '/up',
)
```

La ligne api

Elle indique à Laravel quel fichier charger pour les routes API.

Il s’agit d’un extrait : le reste du fichier est conservé.

Notre première route JSON

Contenu de routes/api.php

```
<?php

use App\Http\Controllers\DocumentExempleController;
use Illuminate\Support\Facades\Route;

Route::get('/exemples/document', [
    DocumentExempleController::class, 'show',
]);
```

Adresse complète : http://localhost:8088/api/exemples/document

Lire la déclaration de route

Comprendre chaque partie de l’exemple

Route::get	La route répond à une demande GET.
Chemin	'/exemples/document' est le chemin défini dans le fichier API.
Contrôleur	DocumentExempleController reçoit la demande.
'show'	La méthode show construit la réponse.

Le rôle d’un contrôleur

Placer le traitement dans une classe PHP

La route

Elle reconnaît la demande et désigne le contrôleur.

Le contrôleur

Sa méthode show prépare les informations à renvoyer.

La réponse

Laravel transforme le tableau PHP en JSON et renvoie un code HTTP.

Créer le contrôleur avec Artisan

Commande exécutée dans le conteneur app

```
docker compose exec app php artisan make:controller DocumentExempleController
```

Artisan

L’outil en ligne de commande de Laravel.

Fichier créé

app/Http/Controllers/DocumentExempleController.php

Dans notre branche de travail, ce fichier existe déjà.

La structure du contrôleur

Fichier DocumentExempleController.php

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Http\JsonResponse;

class DocumentExempleController extends Controller
{
    // Méthode show à placer ici.
}
```

La classe hérite de Controller ; la méthode est ajoutée à l’intérieur.

Extrait de structure : voir la méthode complète sur la page suivante.

La méthode show

La réponse JSON de l’exercice

```
public function show(): JsonResponse
{
    return response()->json([
        'id' => 12,
        'title' => 'Contrat de formation',
        'category' => 'Administratif',
    ]);
}
```

Les trois valeurs sont fictives et écrites dans le contrôleur.

Le fichier choisi dans le formulaire n’est pas envoyé.

Le résultat dans le navigateur

Une réponse HTTP 200 contenant du JSON

http://localhost:8088/api/exemples/document

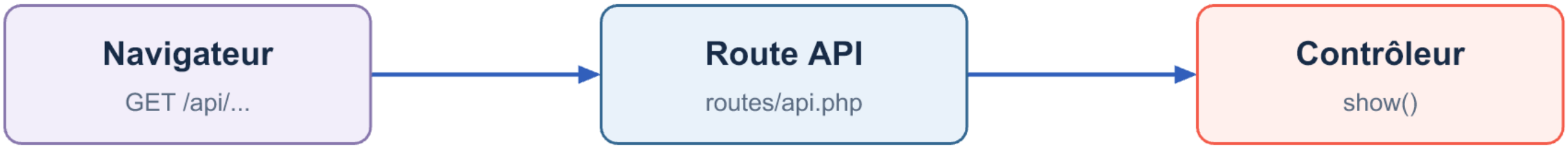

```
{
  "id": 12,
  "title": "Contrat de formation",
  "category": "Administratif"
}
```


Le serveur répond 200 avec Content-Type: application/json.

Ces données ne sont pas encore stockées dans PostgreSQL.

Suivre la requête complète

Du navigateur jusqu'à la réponse du contrôleur



Le contrôleur retourne le JSON ; Laravel le transmet au navigateur.

Aucune lecture en base pour cette première route.

Exercice guidé : créer la route

À réaliser sur une copie de départ où l'exemple n'existe pas

1 Préparer le contrôleur

Dans Docker, créer DocumentExempleController puis ajouter la méthode show.

2 Déclarer la route

Créer routes/api.php avec les imports et la route GET présentés précédemment.

3 Charger le fichier

Ajouter api: __DIR__.'/../routes/api.php' dans bootstrap/app.php.

Suivre les exemples des diapositives ; la correction existe déjà dans la branche de travail.

Exercice guidé : vérifier

Contrôler la route puis ouvrir son adresse

```
docker compose exec app php artisan route:list --path=api
```

Dans la liste

Repérer GET|HEAD et api/exemples/document.

Dans le navigateur

Ouvrir http://localhost:8088/api/exemples/document.

Résultat attendu : un objet JSON et un statut HTTP 200.

Si la route ne fonctionne pas

Trois vérifications pour identifier le problème

Les conteneurs

docker compose ps : app et db doivent être démarrés.

La route

Vérifier routes/api.php, son chargement dans bootstrap/app.php et le préfixe /api.

Le contrôleur

Vérifier le nom de classe, son import et la méthode show.

Pour les journaux : docker compose logs --tail 30 app

Questions de compréhension

Expliquer le rôle de chaque fichier

1 Où se trouve la route GET ?

Quel fichier contient le chemin /exemples/document ?

2 Qui construit la réponse ?

Quelle classe et quelle méthode renvoient le tableau en JSON ?

3 Quel est le rôle de .env ?

Pourquoi ne doit-on pas le publier avec le code ?

Correction

Les éléments à retenir

1 routes/api.php

La route appelle DocumentExempleController::show.

2 Le contrôleur

show utilise response()->json pour construire la réponse.

3 Le fichier .env

Il contient les valeurs locales, dont certaines sont sensibles ; il reste hors Git.

Bilan et prochaine étape

Une première réponse JSON fonctionne

Organisation

Chaque fichier remplit un rôle dans l’application.

Route et contrôleur

Une requête GET appelle show et reçoit du JSON.

Limite de l’exemple

Les informations sont fictives ; PostgreSQL n’est pas encore utilisé pour les documents.

Suite : PostgreSQL, migrations et modèles.

PostgreSQL, migrations et modèles

Comprendre comment Laravel organise les données

La base de données

Comprendre où les informations des documents sont conservées.

La migration

Décrire la structure d’une table et son évolution.

Le modèle

Utiliser Eloquent pour travailler avec les données depuis Laravel.

La pratique détaillée se trouve dans le guide de travaux dirigés.

PostgreSQL et les données

Une base relationnelle organise les informations en tables

Base de données

Un ensemble organisé de données géré par PostgreSQL.



Table

Une structure qui regroupe des enregistrements de même nature.

Ligne et colonne

Une ligne représente un document ; chaque colonne décrit une information.

Une table de documents

Chaque ligne contient les informations d’un document

id	title	category
1	Contrat de formation	Administratif
2	Support de cours	Formation

id distingue chaque ligne. title et category décrivent le document.

Exemple fictif : les valeurs illustrent la structure de la table.

Laravel et la connexion PostgreSQL

Le fichier .env adapte la configuration à chaque poste

Le fichier .env

Il fournit à Laravel les réglages propres à l'environnement local.

La connexion

Le type pgsql et le nom de service db indiquent comment joindre PostgreSQL.

Le volume Docker

Il conserve les données de PostgreSQL après un arrêt ordinaire des conteneurs.

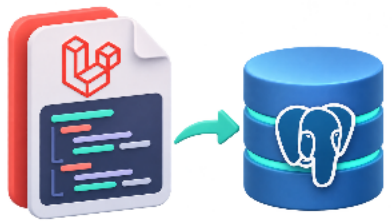
Le guide montre comment vérifier la connexion.

Une migration Laravel

Un fichier qui décrit une évolution de la structure

Son rôle

Définir les changements à apporter à la base, par exemple créer une table.



Dans notre projet

La migration décrit la table documents et les colonnes de ses lignes.

Son intérêt

Toute personne qui récupère le projet peut appliquer la même structure.

Écrire et exécuter une migration

Deux moments différents dans la vie de la base

Écrire le fichier

Le code décrit les changements souhaités. La base reste inchangée.

Exécuter les migrations

Laravel lit les migrations en attente et applique leurs changements à la base configurée.

Suivre les changements

Laravel enregistre les migrations exécutées pour ne pas les rejouer normalement.

Les méthodes up et down

Une migration décrit une modification et son opération inverse

up()

Décrit la modification à appliquer : dans notre cas, créer la table.

down()

Décrit comment revenir en arrière : ici, supprimer la table.

Conséquence

Supprimer une table supprime aussi les données qu’elle contient.

La structure de la table documents

Les instructions de la migration définissent les colonnes

```
$table->id();
$table->string('title', 100);
$table->string('category', 50);
$table->timestamps();
```

Laravel utilise cette description pour créer la table dans PostgreSQL.

Le guide contient le fichier complet et les étapes de création.

Le sens des colonnes

Un identifiant, deux informations et deux dates

id	Identifie chaque document de façon unique.
title	Contient le titre du document.
category	Contient sa catégorie.
timestamps	Ajoute created_at et updated_at.

Ces colonnes décrivent un document : elles ne stockent pas le fichier lui-même.

Le modèle Eloquent

Une classe PHP pour travailler avec les données

Le modèle Document

Il représente les documents manipulés par l'application.

Eloquent

Il permet de lire ou d'enregistrer des objets PHP dans la base.

Une convention Laravel

Par défaut, Document correspond à la table documents et à sa clé id.

Un modèle et une ligne

Chaque objet Document peut représenter un enregistrement

Dans le code PHP

Un objet Document porte des attributs, par exemple son titre et sa catégorie.

Avec Eloquent

Laravel traduit les opérations du modèle en requêtes vers la base.

Dans PostgreSQL

Les valeurs sont conservées dans les colonnes d’une ligne de documents.

Les champs autorisés avec Fillable

Préciser les champs utilisables lors d'une affectation groupée

```
#[Fillable(['title', 'category'])]
```

Dans notre modèle

title et category peuvent être fournis ensemble lors d'une création.

Une distinction

Fillable autorise des champs ; les règles de validation contrôlent leurs valeurs.

Migration et modèle : rôles distincts

Deux éléments complémentaires

Migration

Décrit la table, ses colonnes et leurs types.

Modèle

Permet au code Laravel de lire ou d’enregistrer des documents.

Base PostgreSQL

Conserve les données dans la table créée par la migration.

Du modèle à la base

Le trajet d’une information enregistrée

1 Laravel reçoit des informations

Par exemple, un titre et une catégorie à enregistrer.

2 Eloquent utilise le modèle

Le modèle Document permet de préparer une opération sur la table.

3 PostgreSQL conserve la ligne

La base enregistre les valeurs, qui pourront être relues ensuite.

L’API CRUD sera construite dans la séquence suivante.

Questions de compréhension

Vérifier les notions avant les travaux dirigés

1 Migration

Quel élément décrit les colonnes de documents ?

2 Modèle

Comment Laravel travaille-t-il avec une ligne de documents ?

3 Exécution

Pourquoi la table n’apparaît-elle pas simplement en écrivant la migration ?

Travaux dirigés : documents

Mettre en pratique avec le guide Markdown

Point de départ

La branche de découverte Laravel sert de base pour l’exercice.

Pendant le TD

Le guide accompagne la création des fichiers, la migration et les vérifications.

À la fin

Les participants créent puis relisent un document avec Eloquent.

Guide : TD-Documents-Migration-Modele.md

L’API CRUD des documents

Construire les opérations sur la table documents

Notre objectif

Permettre à un client de lire et modifier les informations des documents.



Notre point de départ

Le modèle Document représente la table préparée pendant le TD précédent.

Notre résultat

Des routes HTTP appellent un contrôleur qui utilise Eloquent.

Les opérations CRUD

Quatre actions sur les informations

Create	Créer une nouvelle ligne de document.
Read	Lire la liste ou consulter un document.
Update	Modifier les informations d’un document.
Delete	Supprimer un document de la base.

Méthodes HTTP et opérations

Une adresse et une méthode identifient l'action

Méthode	Adresse	Action
GET	/api/documents	Lister
GET	/api/documents/{document}	Consulter
POST	/api/documents	Ajouter
PUT	/api/documents/{document}	Modifier
DELETE	/api/documents/{document}	Supprimer

{document} sera remplacé par l'identifiant du document.

Le trajet d’une requête

De la route à la réponse JSON

1 Route

Associe la méthode HTTP et le chemin à une méthode du contrôleur.

2 Contrôleur et modèle

Le contrôleur traite la demande ; Eloquent travaille avec PostgreSQL.

3 Réponse

Laravel renvoie un statut HTTP et, selon le cas, des données JSON.

Créer un document

La requête POST transmet les informations à enregistrer

```
{
  "title": "Contrat de formation",
  "category": "Administratif"
}
```

Dans le contrôleur

Les champs sont validés avant la création de la ligne.

Après création : réponse 201 et document enregistré.

Lister et consulter

Deux lectures pour deux besoins

Lister

GET /api/documents renvoie un tableau, vide ou rempli.

Consulter

GET /api/documents/1 demande le document dont l’identifiant est 1.

Document introuvable

Laravel renvoie une réponse 404 si l’identifiant ne correspond à aucune ligne.

Modifier et supprimer

Les opérations ciblent un document existant

Modifier avec PUT

La demande contient ici le titre et la catégorie, même si un seul champ change.

Supprimer avec DELETE

La ligne est supprimée. La réponse 204 ne contient pas de JSON.

Vérifier

Une nouvelle lecture montre la modification ou signale que la ligne est absente.

La validation des données

Vérifier les valeurs avant leur enregistrement

Titre

Obligatoire, texte, 100 caractères maximum.

Catégorie

Obligatoire, texte, 50 caractères maximum.

Si les données sont invalides

Laravel interrompt le traitement et explique les erreurs dans une réponse 422.

Comprendre les réponses HTTP

Le statut indique le résultat de la demande

200	Lecture ou modification réussie.
201	Document créé.
204	Document supprimé : réponse sans contenu.
404	Document recherché introuvable.
422	Données envoyées invalides.

Travaux dirigés : API CRUD

Construire et tester les cinq routes

Point de départ

La table documents et le modèle ont été préparés au TD précédent.

Pendant le TD

Créer les routes et le contrôleur, puis tester chaque opération et les erreurs.

Après le TD

Comparer le résultat avec la branche de correction.

Guide : TD-API-CRUD-Documents.md

Relier l’interface avec Fetch

Charger et enregistrer les informations des documents

Lire les données

Afficher la liste reçue depuis l’API Laravel.

Envoyer le formulaire

Transmettre le titre et la catégorie avec une requête POST.



Informer l’utilisateur

Afficher une liste vide, une confirmation ou un message d’erreur.

Le rôle de Fetch

Une fonction JavaScript pour échanger avec un serveur

Avant

Le formulaire ajoutait une ligne temporaire dans la page.

Maintenant

Le navigateur demande à Laravel de lire ou enregistrer des informations.

Après la réponse

JavaScript met à jour l’interface avec le résultat ou une erreur.

Le parcours des informations

Du navigateur à PostgreSQL

1 Navigateur

La page envoie une requête GET ou POST à Laravel sur le port 8088.

2 API Laravel

Laravel reçoit la requête et utilise le modèle Document.

3 PostgreSQL et réponse

Laravel lit ou enregistre les données, puis renvoie une réponse JSON.

L’interface est ouverte directement avec index.html.

Charger la liste avec GET

La réponse contient un tableau de documents

Demande

fetch utilise http://localhost:8088/api/documents.

Réponse

Laravel renvoie un tableau JSON ; il peut être vide.

Affichage

JavaScript crée une ligne pour chaque objet reçu.

Attendre une réponse

async et await organisent le code asynchrone

```
async function charger() {
  const response = await fetch('http://localhost:8088/api/documents');
  const documents = await response.json();
}
```

La réponse est reçue avant que les données JSON soient lues.

Le guide ajoute la vérification du statut et la gestion des erreurs.

Vérifier le statut HTTP

Une réponse reçue peut signaler un problème

response.ok

Vaut vrai pour les statuts HTTP compris entre 200 et 299.

Erreur HTTP

Une réponse 422 ou 500 doit être repérée dans le code.

Erreur réseau

La requête peut échouer avant toute réponse : catch permet de l'intercepter.

Comprendre JSON

Un format d’échange entre Laravel et le navigateur

Pour lire

response.json() transforme la réponse en données JavaScript.

Pour envoyer

JSON.stringify transforme le titre et la catégorie en texte JSON.

Les en-têtes

Content-Type décrit le JSON envoyé ; Accept demande du JSON en réponse.

Envoyer le formulaire

Une requête POST avec un corps JSON

```
fetch('http://localhost:8088/api/documents', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json',
    Accept: 'application/json' },
  body: JSON.stringify({ title, category })
});
```

Après succès, la page recharge la liste depuis l’API.

La version complète et la gestion des erreurs se trouvent dans le TD.

Mettre à jour la page

Construire une ligne à partir des données reçues

Pour chaque document

Créer les éléments de la ligne et y placer le titre et la catégorie.

Texte affiché

Utiliser textContent pour afficher les valeurs comme du texte.

Liste vide

Afficher un message clair lorsqu’aucun document n’est enregistré.

Expliquer les erreurs

Informer sans perdre les données saisies

Liste indisponible

La page indique que les documents ne peuvent pas être chargés.

Envoi refusé

La page signale une validation échouée ou un serveur indisponible.

Après une création

Le succès confirme l’enregistrement ; la liste est ensuite rechargée.

Travaux dirigés : Fetch

Relier les deux projets et vérifier le résultat

Point de départ

Laravel et PostgreSQL sont prêts ; le front-end conserve l’affichage compact.

Pendant le TD

Modifier la page et écrire les appels Fetch vers Laravel.

À vérifier

Créer un document, recharger la page et observer les messages d’erreur.

Guide : TD-Fetch-Documents.md